

Web Security Analyzer - Project Plan

Overview

- **Language:** Python
- **Interface:** CLI
- **Output:** JSON + CLI Table
- **Auth Support:** Yes
- **Rate Limiting:** Yes (respects robots.txt, configurable delay)

OWASP Top 10 2025 Coverage

#	Category	Detection Method
A01	Broken Access Control	IDOR testing, forced browsing, path traversal, CORS misconfig
A02	Security Misconfiguration	Missing headers, default pages, debug mode, backup files
A04	Cryptographic Failures	Missing HTTPS, weak TLS, sensitive data in URLs
A05	Injection	SQLi, XSS, Command injection, LDAP injection
A06	Insecure Design	Rate limiting, mass assignment, business logic flaws
A07	Authentication Failures	Weak session tokens, brute force, lockout testing
A09	Security Logging Failures	Stack traces, verbose errors, debug endpoints
A10	Mishandling Exceptions	NULL hints, sensitive data in errors

Project Structure

```
web_analyzer/  
├─ pyproject.toml  
├─ requirements.txt  
├─  
├─ src/web_analyzer/  
│   ├── __init__.py  
│   └─ main.py                # Entry point (Member 4)  
│   └─  
│       └─ core/              # Shared utilities  
│           ├── http_client.py # httpx async wrapper  
│           └─ url_parser.py   # URL normalization
```

```

├── config.py          # CLI arguments
├── robots.py         # robots.txt parser
├── rate_limiter.py    # Request throttling
├── crawler/          # MEMBER 1
│   ├── spider.py     # Main crawler
│   ├── html_parser.py # BeautifulSoup extraction
│   ├── js_parser.py  # JavaScript analysis
│   └── queue.py       # URL deduplication
├── auth/             # Authentication
│   ├── login.py      # Form-based login
│   ├── session.py    # Cookie/token management
│   └── bearer.py     # API key/Bearer auth
├── scanners/         # MEMBER 3
│   ├── base.py       # Scanner base class
│   ├── a01_access_control.py
│   ├── a02_misconfig.py
│   ├── a04_crypto.py
│   ├── a05_injection.py
│   ├── a06_insecure_design.py
│   ├── a07_auth.py
│   ├── a09_logging.py
│   └── a10_exceptions.py
├── output/           # MEMBER 4
│   ├── cli.py        # CLI table output
│   ├── json_report.py # JSON export
│   ├── severity.py   # CVSS scoring
│   └── poc.py        # Proof of concept
├── utils/
│   └── encoder.py     # Base64, URL, HTML encoding
└── tests/
    ├── test_crawler.py
    ├── test_scanners.py
    └── test_output.py

```

pushkar

shruti

Python Dependencies

# Core HTTP & Parsing	
httpx[http2]	# Async HTTP client
beautifulsoup4	# HTML parsing
lxml	# Fast XML/HTML parsing
# Async & Files	
aiofiles	# Async file operations
# CLI & Output	
click	# CLI framework
rich	# Terminal formatting
jinja2	# HTML templates
# Data Validation	
pydantic	# Data models
# Configuration	
python-dotenv	# .env file support

Team Task Distribution

MEMBER 1: Crawler & Discovery Engine

Dependencies: httpx, beautifulsoup4, lxml, aiofiles

File	Responsibilities
core/http_client.py	Async HTTP client with retry, timeout, redirect handling
core/robots.py	Parse and respect robots.txt rules
core/rate_limiter.py	Configurable delay between requests
crawler/spider.py	BFS/DFS crawling, depth control, scope filtering
crawler/html_parser.py	Extract links (<a>), forms (<form>), meta tags
crawler/js_parser.py	Extract API endpoints from JavaScript
crawler/queue.py	URL deduplication, normalization

Deliverable: List[DiscoveredEndpoint] - All URLs, forms, parameters

```
# Output data structure
class DiscoveredEndpoint:
    url: str
    method: str          # GET, POST, etc.
    params: List[Param]
    forms: List[Form]
    headers: Dict
    status_code: int
```

CLI Options Handled:

- --depth : Maximum crawl depth
- --delay : Delay between requests
- --threads : Concurrent request limit
- --exclude : URL patterns to skip
- --user-agent : Custom user agent

MEMBER 2: Authentication Module

Dependencies: httpx , beautifulsoup4

File	Responsibilities
auth/login.py	Detect login forms, auto-submit credentials
auth/session.py	Track cookies, maintain authenticated state
auth/bearer.py	Bearer token, API key support

CLI Options:

```
--auth-form --username USER --password PASS
--auth-bearer "token_here"
--auth-cookies "session=abc123"
```

MEMBER 3: Scanner Engine

Dependencies: httpx , beautifulsoup4 , lxml

A01: Broken Access Control (a01_access_control.py)

Test	Payload/Method
Sequential ID Enumeration	/users/1 , /users/2 , /orders/100
Admin Path Discovery	admin , dashboard , panel , wp-admin , administrator
Path Traversal	../../../../etc/passwd , ../%2F../%2F../%2F
CORS Misconfiguration	Check Access-Control-Allow-Origin: *
Forced Browsing	Access protected paths without auth

A02: Security Misconfiguration (a02_misconfig.py)

Test	Detection
Missing Headers	CSP, X-Frame-Options, HSTS, X-Content-Type
TRACE Method	OPTIONS request with TRACE
Debug Mode	?debug=1 , debug=true
Backup Files	.bak , .swp , ~ , .old
Git Exposure	.git/config , .git/HEAD
Config Exposure	.env , config.php.bak

A04: Cryptographic Failures (a04_crypto.py)

Test	Detection
HTTP Usage	Compare HTTPS vs HTTP responses
TLS Version	Check for TLS 1.0/1.1
Sensitive in URL	Passwords, tokens in GET params
Cookie Flags	Check Secure , HttpOnly

A05: Injection (a05_injection.py)

SQL Injection Tests:

Error-based: ' OR 1=1--
Boolean blind: ' AND 1=1-- / ' AND 1=2--
Time-based: ' AND SLEEP(5)--
Union-based: ' UNION SELECT NULL--

XSS Tests (Reflected):

<script>alert(1)</script>

"><script>alert(1)</script>
<svg onload=alert(1)>

Command Injection:

; ls
| cat /etc/passwd
&& whoami
\$(whoami)
`id`

LDAP Injection:

)(uid=
admin)(&

A06: Insecure Design (a06_insecure_design.py)

Test	Detection
Rate Limiting	Burst requests, check lockout
Mass Assignment	Unexpected param injection
Integer Overflow	Large numbers in numeric fields

A07: Authentication Failures (a07_auth.py)

Test	Detection
Session Token Entropy	Analyze session ID randomness
Default Credentials	Common admin/admin patterns
Missing Lockout	Multiple failed attempts

A09: Security Logging Failures (a09_logging.py)

Test	Detection
Stack Traces	PHP/Java/Python errors
Debug Endpoints	/debug , /actuator , /env
Verbose Errors	Detailed error messages

A10: Mishandling Exceptions (a10_exceptions.py)

Test	Detection
NULL Hints	"NULL pointer", "null value"
Sensitive in Errors	File paths, DB info, stack traces

MEMBER 4: Output & CLI Interface

Dependencies: click , rich , jinja2

File	Responsibilities
output/cli.py	ASCII table with colors, progress bars
output/json_report.py	Structured JSON export
output/severity.py	CVSS 3.1 scoring
output/poc.py	Generate reproducible PoC
core/config.py	CLI argument parsing

CLI Usage

```
# Basic scan
python -m web_analyzer https://example.com
```

```
# With authentication
python -m web_analyzer https://example.com \
    --auth-form --username admin --password pass

# With Bearer token
python -m web_analyzer https://example.com \
    --auth-bearer "eyJhbGciOiJIUzI1NiIs..."

# With cookies
python -m web_analyzer https://example.com \
    --auth-cookies "session=abc123; token=xyz"

# Custom settings
python -m web_analyzer https://example.com \
    --depth 5 \
    --delay 1.0 \
    --threads 10 \
    --exclude "/logout" \
    --output report.json

# Module selection
python -m web_analyzer https://example.com \
    --scanners a01,a05,a07

# Scan specific paths only
python -m web_analyzer https://example.com \
    --scope "/api/*" \
    --scope "/admin/*"

# Verbose output
python -m web_analyzer https://example.com -vv

# Save JSON only (no CLI output)
python -m web_analyzer https://example.com \
    --output report.json \
    --quiet
```

Output Format

CLI Table (Default)

WEB SECURITY SCAN RESULTS			
Vulnerability	Severity	Location	Description
SQL Injection	CRITICAL	POST / login	Boolean-based blind SQLi detected via response diff
Reflected XSS	HIGH	GET / search	Parameter 'q' reflects unsanitized input
Missing CSP Header	MEDIUM	*	Content-Security-Policy header not set
Path Traversal	HIGH	GET / download	/download?file=../../etc/passwd
Debug Mode Enabled	LOW	*	?debug=true parameter accepted

Scan Summary: 5 vulnerabilities found (1 Critical, 2 High, 1 Medium, 1 Low)
Duration: 45.2s | Endpoints: 127 | Target: https://example.com

JSON Output

```

{
  "scan_info": {
    "target": "https://example.com",
    "timestamp": "2025-01-15T10:30:00Z",
    "duration_seconds": 120,
    "endpoints_discovered": 150,
    "scanners_run": ["a01", "a02", "a04", "a05", "a06", "a07", "a09", "a10"]
  },
  "summary": {
    "critical": 1,
    "high": 2,
    "medium": 5,
    "low": 3,
    "info": 10
  },
  "vulnerabilities": [
    {
      "id": "A05-001",
      "category": "A05: Injection",
      "owasp": "A05:2025 Injection",
      "name": "SQL Injection",
      "severity": "CRITICAL",
      "cvss_score": 9.8,
      "location": {
        "url": "https://example.com/login",
        "method": "POST",
        "parameter": "username",
        "context": "form_field"
      },
      "description": "Boolean-based blind SQL injection detected via response differential analysis.",
      "evidence": {
        "original_response_length": 2048,
        "modified_response_length": 2056,
        "differential_detected": true
      },
      "poc": {
        "request": "POST /login HTTP/1.1\nHost: example.com\nContent-Type: application/x-www-form-urlencoded\n\nusername=' OR 1=1--&password=test",
        "response_note": "Response differs when payload is appended"
      },
      "remediation": [
        "Use parameterized queries (prepared statements)",
        "Implement input validation and sanitization",
        "Use an ORM with proper escaping"
      ]
    },
  ],

```

```

    "references": [
        "https://owasp.org/Top10/A05_2025-Injection/",
        "https://portswigger.net/web-security/sql-injection"
    ]
},
{
    "id": "A01-001",
    "category": "A01: Broken Access Control",
    "owasp": "A01:2025 Broken Access Control",
    "name": "Path Traversal",
    "severity": "HIGH",
    "cvss_score": 8.1,
    "location": {
        "url": "https://example.com/download",
        "method": "GET",
        "parameter": "file"
    },
    "description": "Application allows path traversal via 'file'
parameter.",
    "poc": {
        "request": "GET /download?file=../../../../../etc/passwd HTTP/1.1"
    },
    "remediation": [
        "Validate and sanitize user input",
        "Use whitelisting for allowed paths",
        "Implement proper file access controls"
    ]
}
]
}

```

Data Models

Vulnerability

```

from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

```

```

@dataclass

```

```

class Vulnerability:
    id: str # e.g., "A05-001"
    category: str # e.g., "A05: Injection"
    owasp: str # e.g., "A05:2025 Injection"
    name: str # e.g., "SQL Injection"
    severity: str # CRITICAL/HIGH/MEDIUM/LOW/INFO
    cvss_score: float # 0.0 - 10.0
    location: dict # url, method, parameter
    description: str
    evidence: dict
    poc: dict
    remediation: List[str]
    references: List[str]
    timestamp: datetime = None

```

DiscoveredEndpoint

```

@dataclass
class DiscoveredEndpoint:
    url: str
    method: str = "GET"
    params: List[dict] = field(default_factory=list)
    forms: List[dict] = field(default_factory=list)
    headers: dict = field(default_factory=dict)
    status_code: int = 0
    content_type: str = ""
    links: List[str] = field(default_factory=list)
    is_form: bool = False

```

Implementation Timeline

Week	Member 1	Member 2	Member 3	Member 4
1	HTTP Client, Rate Limiter	-	-	CLI skeleton, config
2	Crawler, HTML Parser	-	-	Config parsing
3	JS Parser, Queue	Auth Module	-	Basic output

Week	Member 1	Member 2	Member 3	Member 4
4	Integration testing	-	Base scanner class	-
5	-	-	A05 Injection Scanner	CLI table formatting
6	-	-	A01, A02, A04	Severity scoring
7	-	-	A06, A07, A09, A10	POC generator
8	Full integration + bug fixes			
9	Testing, documentation			

Interface Contracts

Between Crawler and Scanner

```
# crawler/spider.py returns
List[DiscoveredEndpoint] = [
    DiscoveredEndpoint(
        url="https://target.com/api/users",
        method="GET",
        params=[{"name": "id", "value": "1"}],
        forms=[]
    ),
    DiscoveredEndpoint(
        url="https://target.com/login",
        method="POST",
        params=[],
        forms=[{"action": "/login", "inputs": [{"name": "username"},
{"name": "password"}]}]
    )
]

# scanners receive this list and return vulnerabilities
List[Vulnerability] = [
    Vulnerability(
        id="A05-001",
        name="SQL Injection",
        severity="CRITICAL",
```

```
        location={"url": "https://target.com/api/users", "parameter": "id"}
    )
]
```

Between Scanner and Output

```
# scanners return scan results
@dataclass
class ScanResult:
    vulnerabilities: List[Vulnerability]
    scan_info: dict
    endpoints_scanned: int
    duration: float

# output module formats for CLI/JSON
```

Severity Classification

Severity	CVSS Score	Color	Action
CRITICAL	9.0 - 10.0	Red	Immediate action required
HIGH	7.0 - 8.9	Orange	Fix within 24 hours
MEDIUM	4.0 - 6.9	Yellow	Fix within 1 week
LOW	0.1 - 3.9	Blue	Fix within 1 month
INFO	0.0	Gray	Consider fixing

Testing Requirements

Component	Test Coverage
Crawler	URL extraction, deduplication, scope filtering

Component	Test Coverage
Scanners	Known vulnerable test sites (OWASP WebGoat, DVWA)
Auth	Login form detection, session persistence
Output	JSON schema validation, CLI rendering
Integration	End-to-end scan on test targets

File Naming Conventions

```

src/web_analyzer/
├── main.py                # Entry point
├── core/
│   ├── http_client.py    # snake_case
│   ├── url_parser.py
│   ├── config.py
│   ├── robots.py
│   └── rate_limiter.py
├── crawler/
│   ├── spider.py
│   ├── html_parser.py
│   ├── js_parser.py
│   └── queue.py
├── scanners/
│   ├── base.py           # BaseScanner class
│   ├── a01_access_control.py # snake_case for files
│   ├── a02_misconfig.py
│   ├── a04_crypto.py
│   ├── a05_injection.py
│   ├── a06_insecure_design.py
│   ├── a07_auth.py
│   ├── a09_logging.py
│   └── a10_exceptions.py
└── output/
    ├── cli.py
    ├── json_report.py
    ├── severity.py
    └── poc.py

```

Notes for Team

Member 1 (Crawler)

- Focus on robust URL normalization to avoid duplicate requests
- Implement efficient deduplication using URL canonicalization
- Respect robots.txt rules parsed by Member 4's config

Member 2 (Auth)

- Work closely with Member 1 - session should integrate with HTTP client
- Support multiple auth methods: form, bearer, cookies
- Session must persist cookies across all scanner requests

Member 3 (Scanners)

- All scanners inherit from `BaseScanner`
- Use consistent return format: `List[Vulnerability]`
- Implement detection logic first, then optimize for speed
- Each scanner should be independently testable

Member 4 (Output)

- CLI uses `rich` library for colored tables
- JSON output must be valid and schema-consistent
- Integrate all modules in `main.py`
- Handle keyboard interrupts gracefully